Trading Engine: Backtester & Paper Trader

Sharique Ahmad

03 August 2025

1 Introduction

This project is a **quantitative trading framework** designed for both research and real-time testing of algorithmic trading strategies. It provides two modes of operation:

- **Backtesting** – evaluate strategies on historical data.
- **Paper Trading** – simulate live trading on Binance market data without risking capital.

The architecture is modular and extensible, supporting rule-based, machine-learning (ML), and ensemble strategies.

2 Functions of Each File

Each Python file serves a specific role in the framework:

backtester.py

- Implements a fast backtesting engine.
- Supports rule-based and ML-driven strategies.
- Tracks cash, positions, trades, and equity curve.
- Optimized for memory and speed with NumPy arrays.

build_feature.py

- Generates technical indicators from OHLCV data.
- Indicators include SMA, EMA, RSI, ROC, MACD, Bollinger Bands, volatility, and stochastic oscillator.
- Produces clean features for ML models and strategies.

engine.py

- Implements a **matching engine**, similar to an exchange order book.
- Matches buy and sell orders using price-time priority.
- Outputs trade executions when conditions are met.

Note: In this project, the matching engine is not actively used because the execution logic is simplified:

- The backtester and paper trader both assume an *all-in/all-out* model (buy with all cash, or sell entire position).
- This avoids partial fills, order queues, and latency modeling.
- Therefore, execution is immediate and does not require a full order book.

strategy.py

- Defines multiple trading strategies:
  - `MovingAverageStrategy`
  - `MeanReversionStrategy`
  - `EMACrossoverStrategy`
  - `BollingerBandsStrategy`
  - `MomentumStrategy`
  - `MLStrategy` (loads pre-trained ML model)
  - `EnsembleStrategy` (weighted voting among multiple strategies)

Virtual_trader.py

- Implements a paper trading simulator.
- Executes BUY/SELL signals on live Binance data.
- Tracks cash, equity, trades, and prints logs for each trade.

main.py

- Entry point of the system.
- Supports two modes:
  - `backtest`: run simulation with historical data.
  - `paper`: run paper trading with Binance WebSocket API.
- Plots results in real-time (equity curve, buy/sell markers).

train_model.ipynb

- Jupyter notebook for ML model training.
- Saves trained models with their feature columns in `model.pkl`.

3 Feature Engineering and Indicators

The feature builder computes standard technical indicators, used in both ML models and rule-based strategies.

3.1 Simple Moving Average (SMA)

$$SMA_n(t) = \frac{1}{n}\sum_{i=0}^{n-1} P_{t-i}$$

3.2 Exponential Moving Average (EMA)

$$EMA_t = \alpha P_t + (1-\alpha)EMA_{t-1}, \quad \alpha = \frac{2}{n+1}$$

3.3 Relative Strength Index (RSI)

$$RS = \frac{Avg\ Gain}{Avg\ Loss}, \quad RSI = 100 - \frac{100}{1+RS}$$

3.4 Rate of Change (ROC)

$$ROC_n(t) = \frac{P_t - P_{t-n}}{P_{t-n}}$$

3.5 MACD (Moving Average Convergence Divergence)

$$MACD = EMA_{12}(P) - EMA_{26}(P), \quad Signal = EMA_9(MACD)$$

3.6 Bollinger Bands

$$Upper = MA_n + k\sigma_n, \quad Lower = MA_n - k\sigma_n$$

3.7 Volatility

$$\sigma_{t,n} = \sqrt{\frac{1}{n}\sum_{i=0}^{n-1}(r_{t-i} - \bar{r})^2}$$

3.8 Stochastic Oscillator

$$\%K_t = \frac{P_t - L_n}{H_n - L_n} \times 100, \quad \%D_t = SMA_3(\%K)$$

4 Workflow

1. Load data (CSV for backtest, WebSocket for live trading).
2. Build features (if required).
3. Strategy generates signal (`BUY`, `SELL`, `HOLD`).
4. Backtester or paper trader executes trade.
5. Portfolio equity updated.
6. Results visualized in plots and console logs.

5 Setup and Usage

5.1 Dependencies

```
pip install pandas numpy matplotlib joblib scikit-learn python-binance
```

5.2 Backtesting

```
MODE = "backtest"
python main.py
```

5.3 Paper Trading

```
MODE = "paper"
python main.py
```

5.4 ML Training

```
from sklearn.ensemble import RandomForestClassifier
import joblib

model = RandomForestClassifier()
model.fit(X_train, y_train)
joblib.dump((model, list(X_train.columns)), "model.pkl")
```

6 Scope and Assumptions

This framework is designed primarily for **research and testing of trading strategies**. It is not intended for live deployment in production or handling real money.

The system makes several simplifying assumptions about market behavior:

- **Immediate Execution:** All trades are assumed to be filled instantly at the current market price.
- **No Slippage:** There is no price impact from large trades, and no difference between expected and actual execution price.
- **Infinite Liquidity:** The market is assumed to always have enough liquidity to execute trades of any size.
- **No Transaction Costs:** Trading fees, commissions, and taxes are ignored.
- **All-In/All-Out Model:** When buying, all available cash is invested. When selling, the entire position is liquidated.
- **Synchronous Prices:** The framework assumes all features and price updates are synchronized and without delay.

Implications

These assumptions simplify the modeling process and make the framework efficient for backtesting and paper trading. However, they also mean that performance results may not accurately reflect real-world conditions. In practice, factors such as slippage, latency, partial fills, and fees would reduce profitability.

Use Case

The framework should therefore be understood as:

- A **strategy testing tool** for exploring and comparing different algorithms.
- A **learning platform** for algorithmic trading concepts.
- **Not** a production-grade execution system for real-money trading.

7 Conclusion

- Each module has a clear role in the pipeline.
- `engine.py` (matching engine) is not used because this framework uses a simplified execution model.
- The system allows traders to move seamlessly from **backtesting** to **paper trading**.